

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: OBJECT ORIENTED ANALYSIS AND DESIGN
COURSE CODE: CSA1102

COURSE OUTCOME COVERED

CO2: Design UML diagrams to represent system requirements and architecture.
(BL3)

Assessment Tool Weightage: 40%

QUESTIONS: Any 4

Total Marks:40

Annexure A

ANALYTICAL PROBLEM SOLVING

Code of Conduct:

I Sayed Fazal (192311291) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action

Signature of the student:

ANALYTICAL PROBLEM SOLVING

1. Use Case Diagram Problem

A **Hospital Management System** allows patients to book appointments, doctors to update medical records, and administrators to manage hospital resources. The requirements are not clearly documented.

Tasks:

- a) Identify all actors involved in the system.
- b) List the primary use cases.
- c) Define relationships (include, extend, generalization).
- d) Construct a complete Use Case Diagram representing system functionality.

(10 Marks)

Answer:

(a) Actors Involved

The actors interacting with the Hospital Management System are:

1. **Patient**
 - Books appointments
 - Views appointment details
2. **Doctor**
 - Views patient appointments
 - Updates medical records
3. **Administrator**
 - Manages hospital resources
 - Manages doctors and schedules

(b) Primary Use Cases

Patient

- Register/Login
- Book Appointment
- View Appointment
- Cancel Appointment

Doctor

- Login
- View Appointments
- Update Medical Records
- View Patient History

Administrator

- Login
- Manage Hospital Resources
- Manage Doctors
- Manage Appointment Schedule
- Generate Reports

(c) Relationships

<<include>>

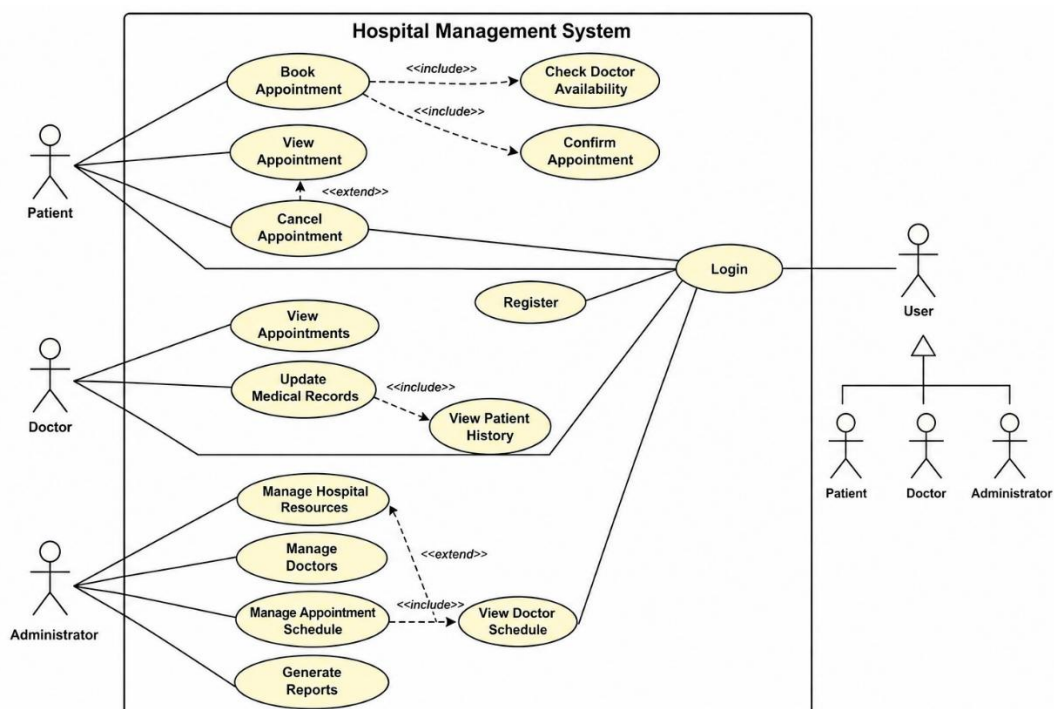
These are mandatory sub-functions.

- **Book Appointment <<include>> Check Doctor Availability**
- **Book Appointment <<include>> Confirm Appointment**
- **Update Medical Records <<include>> View Patient History**
- **Manage Appointment Schedule <<include>> View Doctor Schedule**

<<extend>>

These are optional or conditional behaviors.

- **Cancel Appointment <<extend>> Book Appointment**
- **Generate Reports <<extend>> Manage Hospital Resources**



2. Class Diagram Problem

A **Library Management System** contains entities such as Book, Member, Librarian, and IssueRecord. The relationships among entities are not properly defined.

Tasks:

- a) Identify classes with attributes and methods.
- b) Define relationships (association, aggregation, inheritance).
- c) Specify multiplicity between classes.
- d) Design a detailed Class Diagram for the system.

(10 Marks)

Answer:

(a) Identify Classes with Attributes and Methods

1. Book

Attributes

- bookID : String
- title : String
- author : String
- publisher : String
- isbn : String
- availability : Boolean

Methods

- addBook()
 - updateBook()
 - searchBook()
 - checkAvailability()
-

2. Member

Attributes

- memberID : String
- name : String
- email : String
- phone : String

Methods

- register()
- borrowBook()

- returnBook()
 - viewIssuedBooks()
-

3. Librarian

Attributes

- librarianID : String
- name : String
- designation : String

Methods

- addBook()
 - removeBook()
 - issueBook()
 - receiveBook()
 - generateReport()
-

4. IssueRecord

Attributes

- issueID : String
- issueDate : Date
- dueDate : Date
- returnDate : Date

Methods

- issueBook()
- returnBook()
- calculateFine()

(b) Define Relationships

1. Association

- A **Member** borrows **Book(s)**.
- A **Librarian** manages **Book(s)**.
- A **Member** has **IssueRecord(s)**.

2. Aggregation

- The **Library** (or Library Management System) contains many **Books**.
- Books can exist independently of the library.

3. Inheritance

Create a common superclass **Person**.

(c) Specify Multiplicity Between Classes

Relationship	Multiplicity
Member → IssueRecord	1 : 0..*
Book → IssueRecord	1 : 0..*
Librarian → IssueRecord	1 : 0..*
Library → Book	1 : 1..*
Member → Book	0..* : 0..* (through IssueRecord)

Explanation

- One **Member** can have many **IssueRecords**.
- One **Book** can appear in many **IssueRecords** over time.
- One **Librarian** can issue many books.
- One **Library** contains many books.
- Many members can borrow many books (at different times).

(d) Detailed Class Diagram

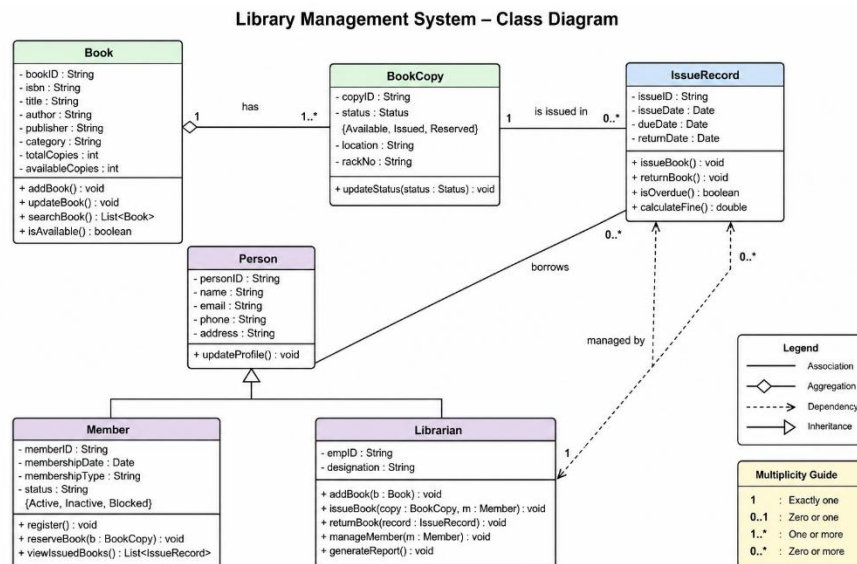


Diagram Summary

- **Classes:** Person, Member, Librarian, Book, IssueRecord
- **Inheritance:** Person → Member, Librarian
- **Association:** Member ↔ IssueRecord ↔ Book
- **Association:** Librarian ↔ IssueRecord
- **Aggregation:** Library ◇— Book

3. Sequence Diagram Problem

In an **Online Food Ordering System**, a customer selects food items, places an order, makes payment, and receives order confirmation. The interaction flow is unclear.

Tasks:

- a) Identify objects involved in the process.
- b) Define message flow between objects.
- c) Represent activation and lifelines.
- d) Construct a Sequence Diagram showing complete interaction flow.

(10 Marks)

Answer:

(a) Identify Objects Involved in the Process

The objects participating in the sequence are:

1. **Customer**
 - Selects food items and places an order.
2. **Food Ordering System**
 - Processes customer requests and coordinates the ordering process.
3. **Restaurant**
 - Receives and prepares the food order.
4. **Payment Gateway**
 - Processes the online payment.
5. **Order Database**
 - Stores order details.
6. **Notification Service**
 - Sends order confirmation to the customer.

(b) Define Message Flow Between Objects

The interaction between the objects occurs in the following sequence:

1. Customer → Food Ordering System: **Browse Menu ()**
2. Customer → Food Ordering System: **Select Food Items ()**
3. Customer → Food Ordering System: **Place Order ()**
4. Food Ordering System → Restaurant: **Send Order ()**
5. Restaurant → Food Ordering System: **Accept Order ()**
6. Food Ordering System → Payment Gateway: **Request Payment ()**
7. Payment Gateway → Food Ordering System: **Payment Successful ()**
8. Food Ordering System → Order Database: **Save Order ()**
9. Order Database → Food Ordering System: **Order Saved ()**
10. Food Ordering System → Notification Service: **Send Confirmation ()**
11. Notification Service → Customer: **Order Confirmation ()**

(c) Represent Activation and Lifelines

Lifelines

Each object has a **lifeline**, represented by a vertical dashed line, showing its existence during the interaction.

The lifelines are:

- Customer
- Food Ordering System
- Restaurant
- Payment Gateway
- Order Database
- Notification Service

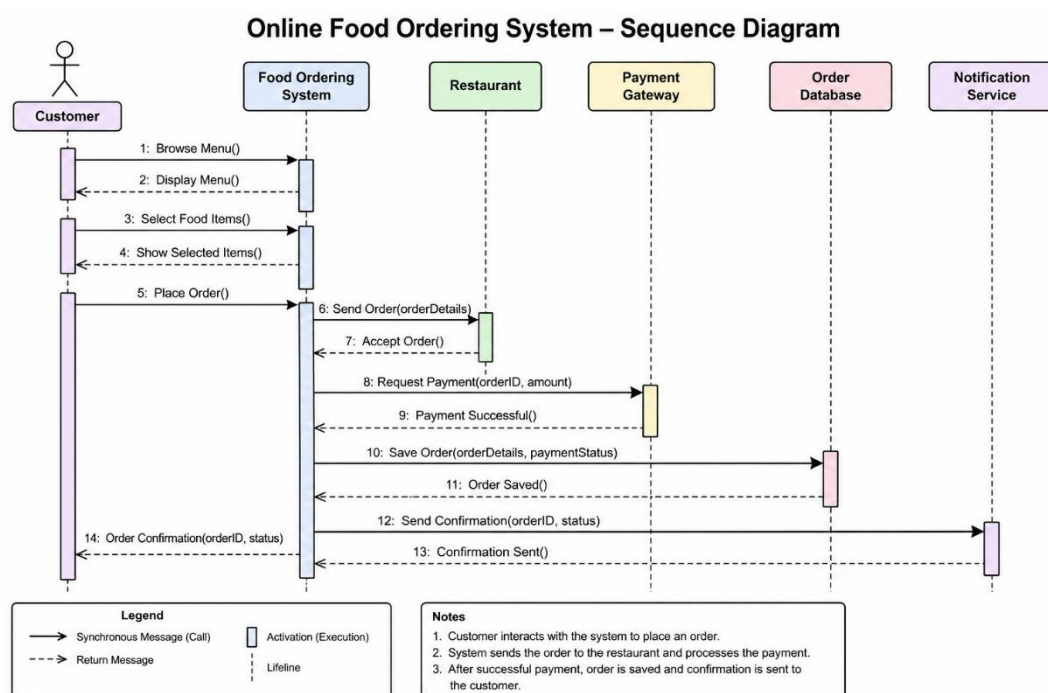
Activation Bars

Activation bars indicate the period during which an object is actively executing an operation.

Examples:

- **Customer** is active while selecting food items and placing the order.
- **Food Ordering System** is active throughout the entire ordering process.
- **Restaurant** is active while accepting the order.
- **Payment Gateway** is active while processing payment.
- **Order Database** is active while saving order details.
- **Notification Service** is active while sending the confirmation message.

d) Construct a Sequence Diagram showing complete interaction flow



4. Activity Diagram Problem

A **Student Admission System** includes application submission, document verification, fee payment, and enrollment confirmation. The workflow lacks proper decision-making steps.

Tasks:

- a) Identify all activities in the workflow.
- b) Add decision nodes for approval/rejection.
- c) Include parallel processing where applicable.
- d) Design an optimized Activity Diagram.

(10 Marks)

Answer:

(a) Identify All Activities in the Workflow

The activities involved in the Student Admission System are:

1. Start
2. Submit Admission Application
3. Upload Required Documents
4. Verify Documents
5. Review Application
6. Approve or Reject Application
7. Pay Admission Fee
8. Verify Fee Payment
9. Generate Enrollment Number
10. Confirm Enrollment
11. End

(b) Add Decision Nodes for Approval/Rejection

The workflow includes the following decision nodes:

Decision 1: Documents Verified?

- **Yes** → Proceed to **Review Application**.
- **No** → Notify student to resubmit documents and return to **Upload Required Documents**.

Decision 2: Application Approved?

- **Yes** → Proceed to **Fee Payment**.
- **No** → Reject application and end the process.

Decision 3: Payment Successful?

- **Yes** → Generate Enrollment Number and confirm admission.
- **No** → Retry payment or cancel the admission process.

(c) Include Parallel Processing Where Applicable

Parallel processing improves the efficiency of the admission process.

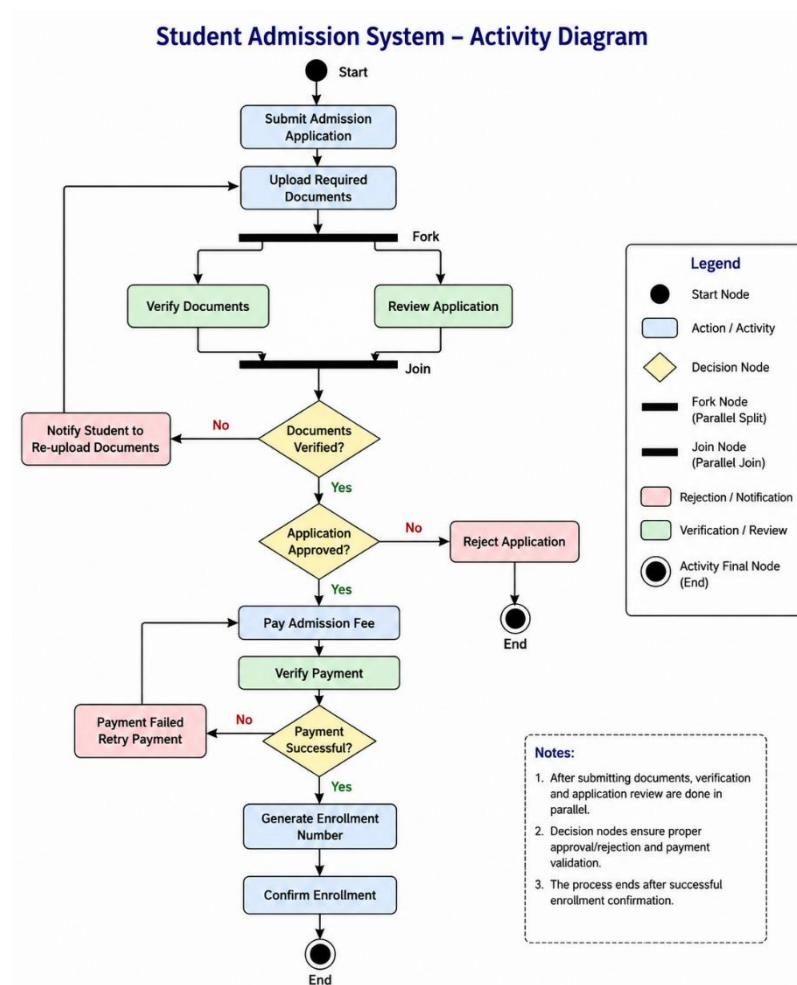
After the student submits the application, the system can perform two activities simultaneously:

- **Document Verification**
- **Application Review**

These activities execute in parallel using a **Fork Node**.

Once both activities are completed successfully, they are synchronized using a **Join Node** before proceeding to the approval decision.

(d) Design an Optimized Activity Diagram



RUBRICS FOR CALCULATION

ANALYTICAL PROBLEM SOLVING

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Understanding	20%	Clearly interprets problem, identifies all parameters and requirements accurately	Understands problem with minor gaps	Partial understanding; misses key elements	Misinterprets or unable to understand problem
Method Selection	20%	Selects most appropriate method/formula with strong justification	Selects correct method with minor errors	Inappropriate or partially correct method	Unable to select method
Solution Process	25%	Logical, systematic steps with correct approach throughout	Mostly correct steps with minor mistakes	Disorganized steps; multiple errors	No clear process
Accuracy of Calculation	20%	All calculations accurate; correct final answer	Minor calculation errors	Frequent errors; incorrect final answer	No valid calculations
Interpretation of Results	15%	Clearly interprets results with units and engineering relevance	Basic interpretation with minor omissions	Limited or unclear interpretation	No interpretation